ELI — Learning

ELI v2.1.65

August 2026

Contents

ELI Learning — LoRA Self-Training Pipeline

`eli/learning/` — 3.3k LOC, 12 files. ELI's self-improvement-via-fine-tuning path. **Important framing:** this is a *curated, human-gated, operator-invoked* training pipeline — **not** an autonomous self-modifying loop, and it targets a **separate trainable base (Phi-3)**, not the inference GGUF.

Files & the pipeline

The stages, roughly in order:

File	LOC	Stage
`dataset_builder.py`	558	turn ELI's logged turns/corrections into supervised examples
`dataset_filters.py`	154	quality gates (`is_bad_response`, `row_is_reviewed`)
`export_trainable_dataset.py`	168	export reviewed rows → trainable JSONL
`merge_reviewed_datasets.py`	135	merge reviewed dataset shards
`bootstrap_phi3_base.py`	245	one-time download of the trainable HF Phi-3 base
`base_model_resolver.py`	185	locate/validate the trainable base (vs an adapter)
`training_preflight.py`	142	check peft/transformers/datasets present + target ready
`lora_trainer_guard.py`	571	`TrainerTarget` plans (paths, base_family) + guard checks
`lora_trainer.py`	656	
`lora_eval.py`	491	eval harness (score vs expected/forbidden, inspect adapter)

Flow

1. **Build** (`dataset_builder.py`): mines ELI's own conversation/correction history into `SupervisedExamples`. Crucially includes `clean_text` + **`redact_text` (PII redaction)** and an exclusion set so sensitive/garbage content never becomes a training example. Writes `training/datasets/eli_supervised_v0.jsonl` + a report.
2. **Gate** (`dataset_filters.py`): `is_bad_response` and `row_is_reviewed` enforce quality + a **human review flag**.
3. **Export / merge**: reviewed rows → `*.trainable.jsonl`.
4. **Preflight** (`training_preflight.py`): refuses to proceed unless the HF training stack is installed and the target/base resolve.
5. **Train** (`lora_trainer.py`): `_dataset_report` **refuses to train** on a dataset with unreviewed rows, wrong-target rows, or bad-response rows — i.e. it will not learn from un-vetted data. Loads Phi-3 through native transformers (with a RoPE-scaling compat shim), trains the LoRA adapter.
6. **Eval** (`lora_eval.py`): `score_response` against expected/forbidden, plus `inspect_adapter` / `inspect_eval_suite`.

Targets (`lora_trainer_guard.py`)

`TrainerTargets` define adapter/dataset/output paths for `phi3` and `phi3-ultra` variants under `models/lora/adapters/`. `base_family` is parameterised (`phi3`/`mistral`/`qwen`), so the trainer isn't hard-locked to one family even though Phi-3 is the default trainable base.

What it is — and isn't

- It **is**: a responsible, reproducible fine-tuning pipeline with PII redaction, human review gating, preflight, and an eval harness. The dataset comes from ELI's real interactions (corrections, failures, conversations).
- It **isn't**: (a) automatic — no runtime trigger wires it in; it's run via CLI/`main()` entrypoints (`bootstrap_phi3_base`, `training_preflight`, `lora_trainer`). (b) connected to the live inference model — it trains a Phi-3 adapter, while inference typically runs a different GGUF. So a trained improvement does **not** flow into the running assistant unless you also run the Phi-3 base + adapter as the inference model.

Honest assessment

- **Strong (and unusually responsible):** PII redaction, refusing to train on unreviewed/bad rows, a preflight, and an eval suite are exactly what a serious fine-tuning loop needs and what most hobby projects skip. The base-vs-adapter validation prevents the classic "trained on an adapter" mistake.
- **Weak / watch:**
  1. **Base/inference disconnect** — training improves Phi-3, but the assistant usually runs another GGUF, so the self-improvement doesn't reach the live model. The loop is only closed if you actually serve the trained model.
  2. **Manual** — "self-training" is operator-driven, not autonomous. That's safe, but it means it improves only when you run it. (Arguably the right trade-off, but worth stating plainly rather than implying autonomy.)
  3. **Heavy deps** — needs the full HF/peft/transformers stack present; the preflight handles absence gracefully, but it's a large optional surface.
  4. **Phi-3 default** — intentional and parameterised, but the canned `TrainerTargets` and several resolver candidates are Phi-3-named, so adding a new base family is more than a config change.